Object Oriented Programming: Exercises

1.- Modify class GradeBook (Fig. 3.10) as follows:

a) Include a second String instance variable that represents the name of the course's instructor.
b) Provide a *set* method to change the instructor's name and a *get* method to retrieve it.
c) Modify the constructor to specify two parameters—one for the course name and one for the instructor's name.
d) Modify method displayMessage such that it first outputs the welcome message and course name, then outputs "This course is presented by: " followed by the instructor's name.

Use your modified class in a test application that demonstrates the class's new capabilities.

2.- Create a class called Invoice that a hardware store might use to represent an invoice for an item sold at the store. An Invoice should include four pieces of information as instance variables—a part number (type String), a part description (type String), a quantity of the item being purchased (type int) and a price per item (double). Your class should have a constructor that initializes the four instance variables. Provide a *set* and a *get* method for each instance variable. In addition, provide a method named getInvoiceAmount that calculates the invoice amount (i.e., multiplies the quantity by the price per item), then returns the amount as a double value. If the quantity is not positive, it should be set to 0. If the price per item is not positive, it should be set to 0.0. Write a test application named InvoiceTest that demonstrates class Invoice's capabilities.

3.- Create a class called Employee that includes three pieces of information as instance variables— a first name (type String), a last name (type String) and a monthly salary (double). Your class should have a constructor that initializes the three instance variables. Provide a *set* and a *get* method for each instance variable. If the monthly salary is not positive, set it to 0.0. Write a test application named EmployeeTest that demonstrates class Employee's capabilities. Create two Employee objects and display each object's *yearly* salary. Then give each Employee a 10% raise and display each Employee's yearly salary again.

4.- Create a class called Date that includes three pieces of information as instance variables—a month (type int), a day (type int) and a year (type int). Your class should have a constructor that initializes the three instance variables and assumes that the values provided are correct. Provide a *set* and a *get* method for each instance variable. Provide a method displayDate that displays the month, day and year separated by forward slashes (/). Write a test

application named DateTest that demonstrates class Date's capabilities.

5.- *(Rectangle Class)* Create a class Rectangle. The class has attributes length and width, each of which defaults to 1. It has methods that calculate the perimeter and the area of the rectangle. It has *set* and *get* methods for both length and width. The *set* methods should verify that length and width are each floating-point numbers larger than 0.0 and less than 20.0. Write a program to test class Rectangle.

6.- *(Savings Account Class)* Create class SavingsAccount. Use a static variable annualInterestRate to store the annual interest rate for all account holders. Each object of the class contains a private instance variable savingsBalance indicating the amount the saver currently has on deposit. Provide method calculateMonthlyInterest to calculate the monthly interest by multiplying the savingsBalance by annualInterestRate divided by 12—this interest should be added to savings- Balance. Provide a static method modifyInterestRate that sets the annualInterestRate to a new value. Write a program to test class SavingsAccount. Instantiate two savingsAccount objects, saver1 and saver2, with balances of $2000.00 and $3000.00, respectively. Set annualInterestRate to 4%, then calculate the monthly interest and print the new balances for both savers. Then set the annualInterestRate to 5%, calculate the next month's interest and print the new balances for both savers.

7.- *(Enhancing Class Time2)* Modify class Time2 of Fig. 8.5 to include a tick method that increments the time stored in a Time2 object by one second. Provide method incrementMinute to increment the minute and method incrementHour to increment the hour. The Time2 object should always remain in a consistent state. Write a program that tests the tick method, the increment- Minute method and the incrementHour method to ensure that they work correctly. Be sure to test the following cases:

a) incrementing into the next minute,
b) incrementing into the next hour and
c) incrementing into the next day (i.e., 11:59:59 PM to 12:00:00 AM).

8.- *(Enhancing Class Date)* Modify class Date of Fig. 8.7 to perform error checking on the initializer values for instance variables month, day and year (currently it validates only the month and day). Provide a method nextDay to increment the day by one. The Date object should always remain in a consistent state. Write a program that tests the nextDay method in a loop that prints the date during each iteration of the loop to illustrate that the nextDay method works correctly. Test the following cases:

a) incrementing into the next month and

b) incrementing into the next year.

9.- *(Huge Integer Class)* Create a class HugeInteger which uses a 40-element array of digits to store integers as large as 40 digits each. Provide methods input, output, add and subtract. For comparing HugeInteger objects, provide the following methods: isEqualTo, isNotEqualTo, isGreaterThan, isLessThan, isGreaterThanOrEqualTo and isLessThanOrEqualTo. Each of these is a predicate method that returns true if the relationship holds between the two HugeInteger objects and returns false if the relationship does not hold. Provide a predicate method isZero. If you feel ambitious, also provide methods multiply, divide and remainder. [*Note:* Primitive boolean values can be output as the word “true” or the word “false” with format specifier %b.]